

Архитектурно предложение за проект Skills Barter

Автори: Боян Венциславов Фердинандов, Мехмед Салих Мустафов, Пламен Кирилов Николов, Слави Валериев Митов, Станимир Костов Райков

Въведение

Този документ представя архитектурното предложение за проекта Skills Barter и служи като обща рамка за планиране, комуникация и вземане на решения в екипа. Адресира основните заинтересовани страни - възложител/преподавател, архитект, разработчици, тестови екип и осигурява очаквания за функционалност, качество и интеграции. Авторы на документа са Боян Венциславов Фердинандов, Мехмед Салих Мустафов, Пламен Кирилов Николов, Слави Валериев Митов и Станимир Костов Райков, с ясно разпределени роли по раздели (Use-case/Актьори – Пламен Логически изглед – Слави, Процесен изглед – Мехмед, Изглед на данните – Боян, Изглед на внедряването – Станимир).

Обхватът покрива концептуални и логически решения, модели на данни, основни процеси и нефункционални изисквания (сигурност, производителност, разширяемост, тестване, интероперабилност и използваемост), както и технологичен стек и външни интеграции. Документът не описва детайлите по имплементация и оперативни процедури извън дефинираната архитектура, но задава критерии за приемане и проследимост между бизнес процеси и технически компоненти. Целта е да осигури последователна, проверима и реалистична основа за последваща детайлизация, разработка и внедряване.

Предназначение

Обхват

Тук се посочва каква част от жизнения цикъл на проекта се обхваща от този документ.

Актьори

В тази секция са представени участниците (актьорите), заинтересовани от архитектурния документ на проекта **Skills Barter**, както и частите от него, които ги засягат пряко. Всеки актьор играе конкретна роля в жизнения цикъл на системата

– от планирането и проектирането, през реализацията, до поддръжката и използването ѝ.

Актьор	Роля в проекта	Интерес / Отговорности в документа
Крайни потребители (Users)	Използват платформата за обмен на умения и услуги	Отнася се до функционалните изисквания, интерфейса (UX/UI) и сигурността на личните данни.
Администратори / Модератори	Управляват съдържание, ревюта и потребителски профили	Касае ги архитектурата на бекенда, контролът на достъпа и ролевата система.
Разработчици (Fullstack екип)	Отговарят за изграждането и поддръжката на системата	Пряко ангажирани с архитектурния, логическия и процесния изглед на документа.
UI/UX дизайнери	Проектират потребителския интерфейс и изживяване	Интерес към изгледа „Използваемост“ и визуалните компоненти на фронтенда.
DevOps / Системни администратори	Внедряване и поддръжка на инфраструктурата	Заинтересовани от изгледа на внедряване (deployment view) и секцията с нефункционални изисквания.

Маркетинг екип / Бизнес анализатори	Следят за позиционирането на продукта и обратната връзка	Интерес към интеграциите с външни системи и аналитичните модули (Google Analytics, PayPal и др.).
Тестери / QA инженери	Провеждат тестове и гарантират качеството на системата	Отнася се до частта за „Възможност за тестване“ и автоматизацията на тестовете.

Използвани термини и символи

Използвани термини и символи – всички

В този раздел са представени основните термини, понятия и съкращения, използвани в архитектурния документ на проекта **Skills Barter**. Те имат за цел да осигурят яснота и последователност при тълкуването на концепциите и описанията в останалите раздели.

Термин / Понятие	Описание
Бартер	Вид обмен между двама или повече потребители, при който се заменят услуги или умения, без използване на парични средства.
Умения (Skills)	Практически способности или компетенции, които потребителят може да предложи на други потребители.

Оферта (Offer)	Обява, публикувана от потребител, която описва услуга или умение, достъпно за бартер.
Заявка / Искане (Request)	Комуникационен елемент между потребители за договаряне на конкретен обмен.
Споразумение (Agreement)	Финален етап от бартерния процес, при който участниците се съгласяват върху условията на обмена.
Ревю / Отзив (Review)	Текстова оценка, която потребителят оставя след успешно приключен обмен.
Рейтинг (Rating)	Числова стойност, която отразява репутацията на потребителя в системата.
Роля (Role)	Определя нивото на достъп и възможностите на потребителя в системата (Freemium, Premium, Moderator).
Потребител (User)	Всеки регистриран участник в платформата, който може да създава, търси или приема оферти.
Модератор / Администратор	Потребители със специални права за контрол на съдържание, сигурност и спазване на правилата.
Нотификация (Notification)	Автоматично съобщение, което информира потребителя за промени, нови съобщения или ревюта.
Каталог на умения (Skills Catalog)	Организирана колекция от категории и подкатегории, които описват видовете умения в системата.

Съкращение / Символ	Значение
API – Application Programming Interface	Интерфейс за комуникация между фронтенда и бекенда.
JWT – JSON Web Token	Токен, използван за удостоверяване на потребителската сесия.
REST – Representational State Transfer	Архитектурен стил за реализиране на уеб услуги.
DB – Database	База данни (PostgreSQL), съхраняваща потребителска и системна информация.
UI / UX – User Interface / User Experience	Потребителски интерфейс / потребителско изживяване.
SMTP – Simple Mail Transfer Protocol	Протокол за изпращане на имейл съобщения и верификации.
OAuth2	Протокол за външна автентикация (чрез Google, Facebook и др.).
HTTPS	Сигурен протокол за пренос на данни между клиент и сървър.

Източници

<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-implementation-entity-framework-core>

<https://devguide.owasp.org/en/06-verification/01-guides/03-asvs/>

Архитектурен обзор

Тук се посочва чрез какви изгледи е представена архитектурата и каква информация носи всеки от тях. Всеки от изгледите се представя с кратко описание (каква е неговата цел и обхват), а в следващите глави на документа и подробно (чрез диаграми и описателен текст).

**Употребата на конкретни изгледи не е задължителна, но води до яснота и структуриран подход за реализация на проекта. Посочените изгледи по-долу са подредени спрямо честотата им на срещане.*

В зависимост от реализацията могат да се включат изгледи като:

1. Use-case изглед – представя функционалните възможности на софтуера (основни сценарии) от гледна точка на ползвателите (актьори). Използват се диаграми (*UML Use-case diagrams*) и текстово описание на функциите; **(описване на процеси от клиентска гледна точка)**

Use-case изгледът представя функционалните възможности на софтуера **Skills Barter** от гледна точка на основните актьори – крайните потребители, модераторите и администраторите.

Целта е да се опишат ключовите сценарии на взаимодействие между потребителя и системата, както и основните потоци от действия, които осигуряват бартерния процес.

Основни актьори

1. **Потребител (User)** – основен участник, който може да предлага или търси услуги, да комуникира с други потребители и да участва в обмен.

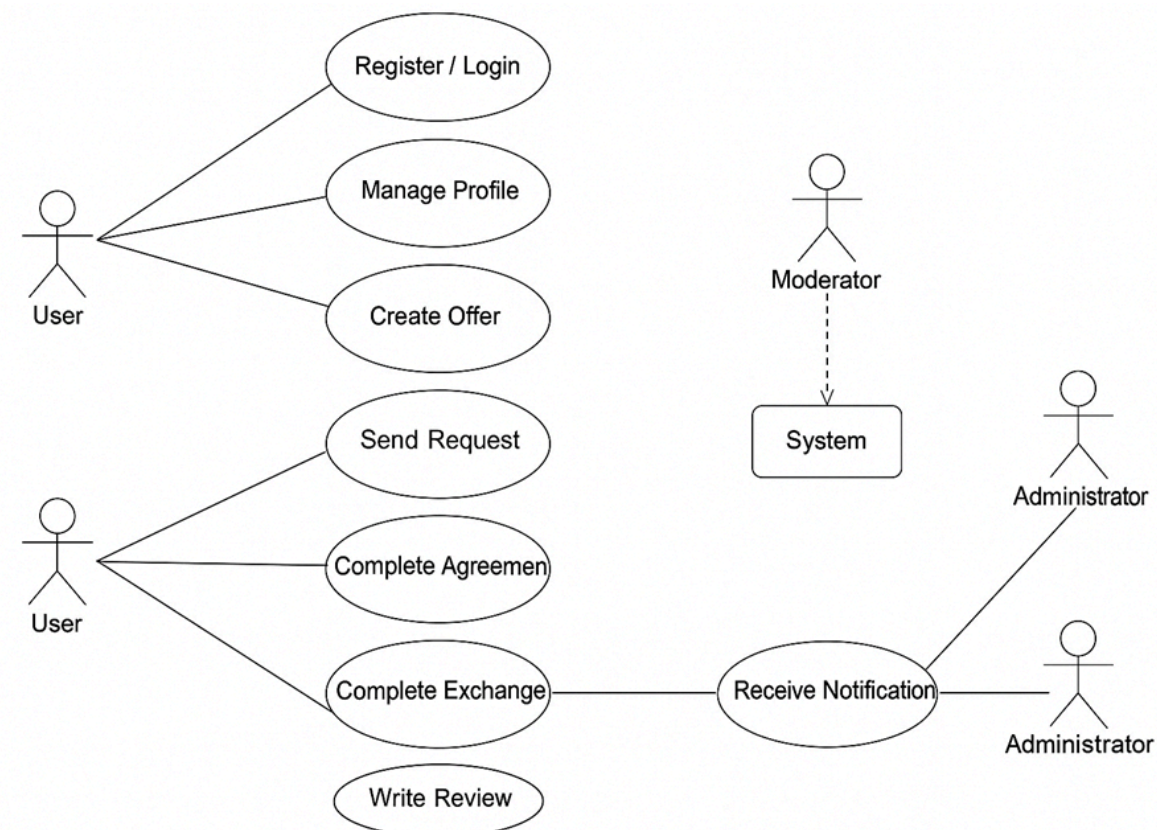
Типове потребителски акаунти:

Freemium User – стандартен потребител с ограничен брой активни оферти.

Premium User – потребител с платен абонамент, който има право да публикува по-голям брой активни оферти.

2. **Модератор (Moderator)** – следи за спазването на правилата, валидира оферти и може да премахва неподходящо съдържание.

3. **Администратор (Administrator)** – управлява потребители, роли, абонаменти и глобални настройки на системата.



2. Логически изглед

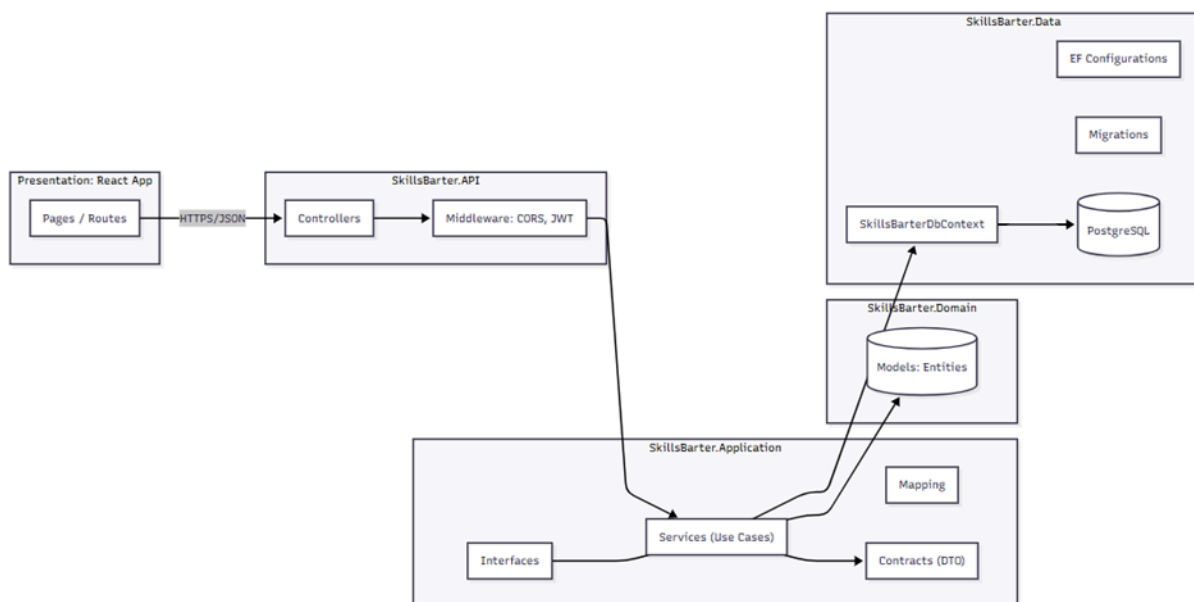
2.1 Слоеве и зависимости

- SkillsBarter.API - Controllers, Program.cs, appsettings.json.
- SkillsBarter.Application - Services (use cases), Interfaces (интерфейси на services), Mapping.
- SkillsBarter.Domain – Models (Entities/ValueObjects), Contracts (DTO).
- SkillsBarter.Data – DbContexts (EF Core + Npgsql), Migrations.
 - Услугите работят директно с SkillsBarterDbContext. Това е валидно и поддържано от Microsoft: DbContext е базиран върху Unit of Work и Repository шаблони и може да се ползва директно за прост и чист код.

Инфраструктурни политики (в API):

- JWT Bearer за автентикация/авторизация
- CORS allowlist към React origin-и.

Сроевият стил е съвместим с препоръките „Common web application architectures / Clean architecture“.



2.2 Примерна структура на solution-a

SkillsBarter.sln

SkillsBarter.API/

 Controllers/

 AuthenticationController.cs

 UsersController.cs

 SkillsController.cs

 CategoriesController.cs

 OffersController.cs

 ThreadsController.cs

 MessagesController.cs

 AgreementsController.cs

 MilestonesController.cs

 ReviewsController.cs

 DisputesController.cs

 NotificationsController.cs

 Program.cs

 appsettings.json

SkillsBarter.Application/

 Interfaces

 IAuthService.cs

 IUsersService.cs

 ISkillsService.cs

 IOffersService.cs

 IThreadsService.cs

 IMessagesService.cs

 IAgreementsService.cs

 IMilestonesService.cs

 IReviewsService.cs

 IDisputesService.cs

 INotificationsService.cs

 Services/

 Auth/

 AuthService.cs

 TokenRefreshService.cs

 Users/

 UsersService.cs

- Skills/
 - SkillsService.cs
 - CategoriesService.cs
 - Offers/
 - OffersService.cs
 - OfferSearchService.cs
 - Threads/
 - ThreadsService.cs
 - MessagesService.cs
 - Agreements/
 - AgreementsService.cs
 - MilestonesService.cs
 - Reviews/
 - ReviewsService.cs
 - Disputes/
 - DisputesService.cs
 - Mapping/
 - OfferMappingProfile.cs
 - ThreadMappingProfile.cs
 - AgreementMappingProfile.cs
 - UserMappingProfile.cs
 - Contracts/
 - Users/ (ProfileFormModel, ProfileDisplayModel, ...)
 - Skills/ (SkillFormModel, SkillDisplayModel, SkillCategoryDisplayModel, ...)
 - Offers/ (OfferFormModel, OfferDisplayModel, ...)
 - Threads/ (ThreadDisplayModel, MessageFormModel, MessageDisplayModel, ...)
 - Agreements/ (AgreementFormModel, AgreementDisplayModel, ...)
 - Milestones/ (MilestoneFormModel, MilestoneDisplayModel, ...)
 - Reviews/ (ReviewFormModel, ReviewDisplayModel, ...)
 - Disputes/ (DisputeFormModel, DisputeMessageFormModel, ...)
- SkillsBarter.Domain/
 - Models/
 - User.cs
 - Profile.cs
 - UserRole.cs
 - SkillCategory.cs
 - Skill.cs
 - Offer.cs

OfferStatus.cs
OfferTag.cs
Thread.cs
Message.cs
Agreement.cs
Milestone.cs
Review.cs
Dispute.cs
DisputeMessage.cs
Notification.cs

Contracts/

SkillsBarter.Data/
 DbContexts/
SkillsBarterDbContext.cs
 Migrations/
 ...

Services имплементират интерфейси и инджектват директно SkillsBarterDbContext (EF Core).

2.3 Домейн модел

UML диаграма и по-подробно описание в **точка 4. Изглед на данните**

правила:

Offer агрегира OfferTag (валидни статуси: draft/active/paused/closed).

Thread агрегира Message (само участници могат да пишат).

Agreement агрегира Milestone (валиден статус флоу; ProviderId, ClientId).

Review: най-много 1 рецензия на участник за даден Agreement.

Dispute агрегира DisputeMessage (участници/модератори).

2.4 Application слой

примери:

```
// Application/Interfaces/IOffersService.cs
public interface IOffersService {
    Task<Guid> CreateAsync(OfferFormModel dto, Guid actorId);
    Task ChangeStatusAsync(Guid offerId, string newStatus, Guid actorId);
    Task<IReadOnlyList<OfferDisplayModel>> SearchAsync(OfferSearchModel query);
}
```

```
// Application/Services/Offers/OffersService.cs
public sealed class OffersService : IOffersService {
    private readonly SkillsBarterDbContext _db;
    public OffersService(SkillsBarterDbContext db) => _db = db;

    public async Task<Guid> CreateAsync(OfferFormModel dto, Guid actorId) {
        var offer = new Offer {
            OwnerId = actorId,
            SkillId = dto.SkillId,
            Title = dto.Title,
            Description = dto.Description,
            StatusCode = "active",
            CreatedAt = DateTime.UtcNow,
            UpdatedAt = DateTime.UtcNow
        };
        _db.Offers.Add(offer);
        await _db.SaveChangesAsync();
        return offer.Id;
    }

    public Task ChangeStatusAsync(Guid id, string status, Guid actorId) {
        // ... зареждане, проверки на ownership и валиден статус, запис чрез _db
        throw new NotImplementedException();
    }
}
```

```

    public async Task<IReadOnlyList<OfferDisplayModel>>
SearchAsync(OfferSearchModel q) {
    // ... LINQ към _db.Offers.Include(...).Where(...).Select(...).ToListAsync();
    return new List<OfferDisplayModel>();
}
}

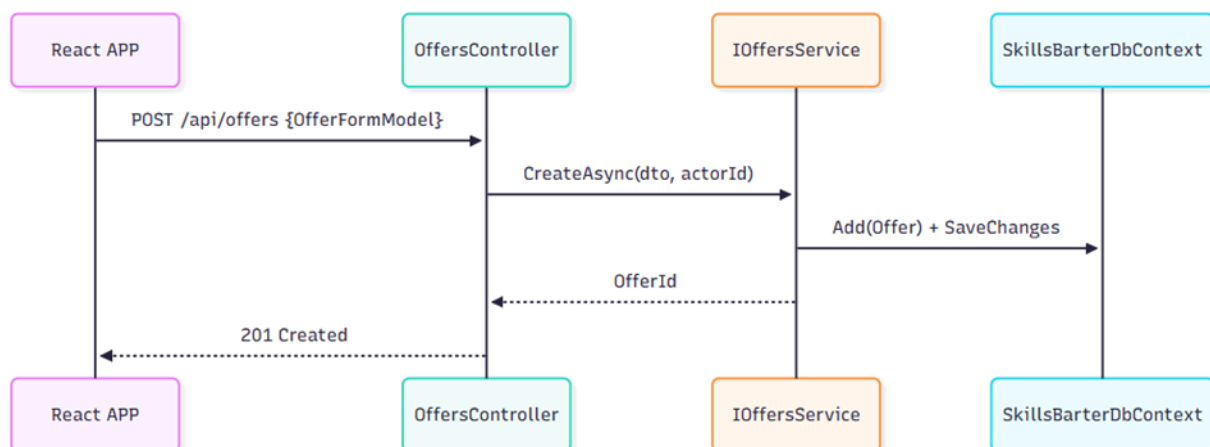
```

2.5 API слой – контролери и DTO конвенции

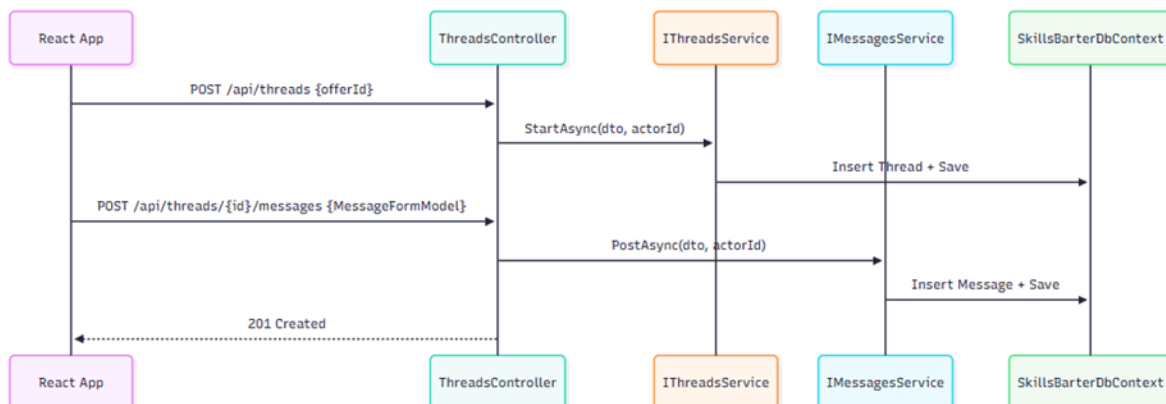
- Именуване: XxxController.cs (мн. число за ресурсите), входни модели *FormModel, изходни *DisplayModel, заявки.
- Групи endpoints:
 - /auth/* – login/refresh/external (OIDC), издаване на JWT.
 - /users, /profiles, /skills, /categories
 - /offers, /threads, /messages
 - /agreements, /milestones, /reviews, /disputes, /notifications
- Middleware:
 - CORS – allowlist на фронтенд origin-и (React SPA).
 - JWT Bearer – валидиране на issuer/audience/exp/signature.

Последователности (примерни диаграми)

Create Offer:



Thread & Message:



2.6 Data слой - EF Core + Npgsql

DbContext: SkillsBarterDbContext с DbSet<> за всяка entity;

Fluent конфигурации в Data/Configurations.

Миграции: dotnet ef migrations add ... -> Data/Migrations.

Защо директен DbContext е окей: Microsoft изрично отбелязва, че DbContext е базиран на Unit of Work/Repository и може да се използва директно за най-четим код, без допълнителни слоеве

2.7 Нефункционални опори

JWT Bearer (в API): конфигурация с AddAuthentication().AddJwtBearer(...); валидира iss/aud/exp; контролери с [Authorize].

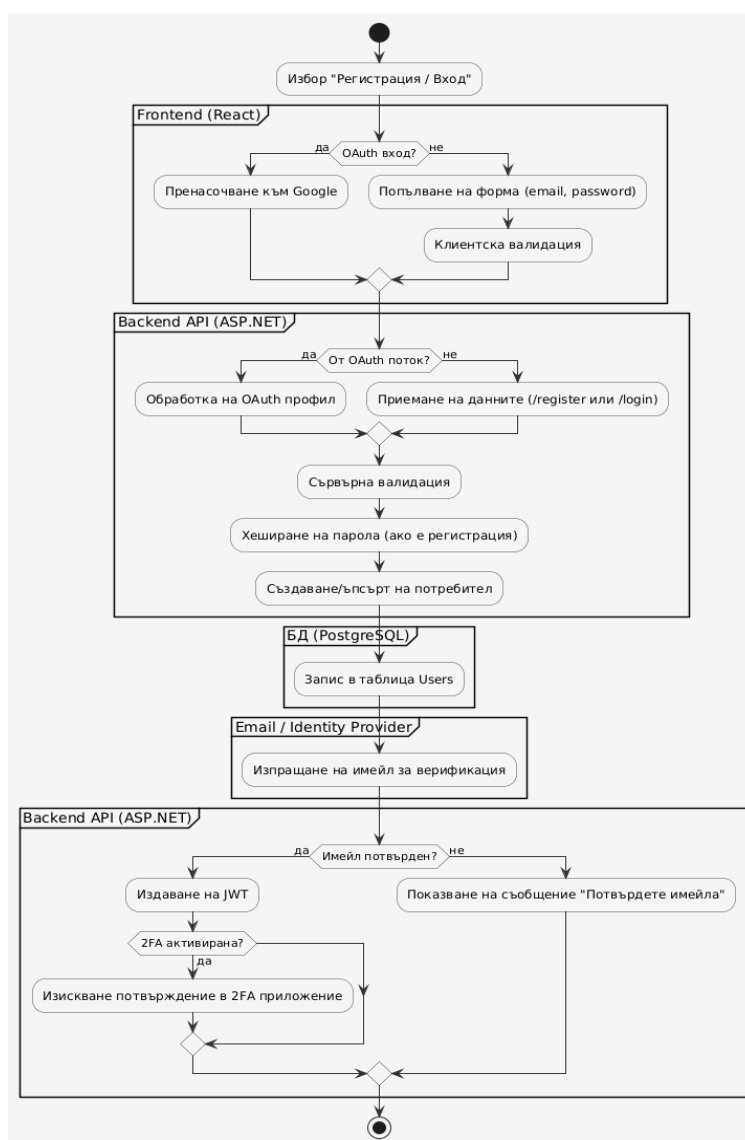
- CORS: именувана политика (напр. "Frontend"), allow-лист за React origin-и, UseCors("Frontend").

Словен стил/Clean: бизнес логиката в Application/Domain, инфраструктурата зависи от тях (dependency inversion)

3. Процесен изглед – представя алгоритмите, реализиращи архитектурно значимите функции. Използват се диаграми (*UML activity diagrams*) в комбинация с текстово описание на действията; (**развитие на методите написани в точка 2**)

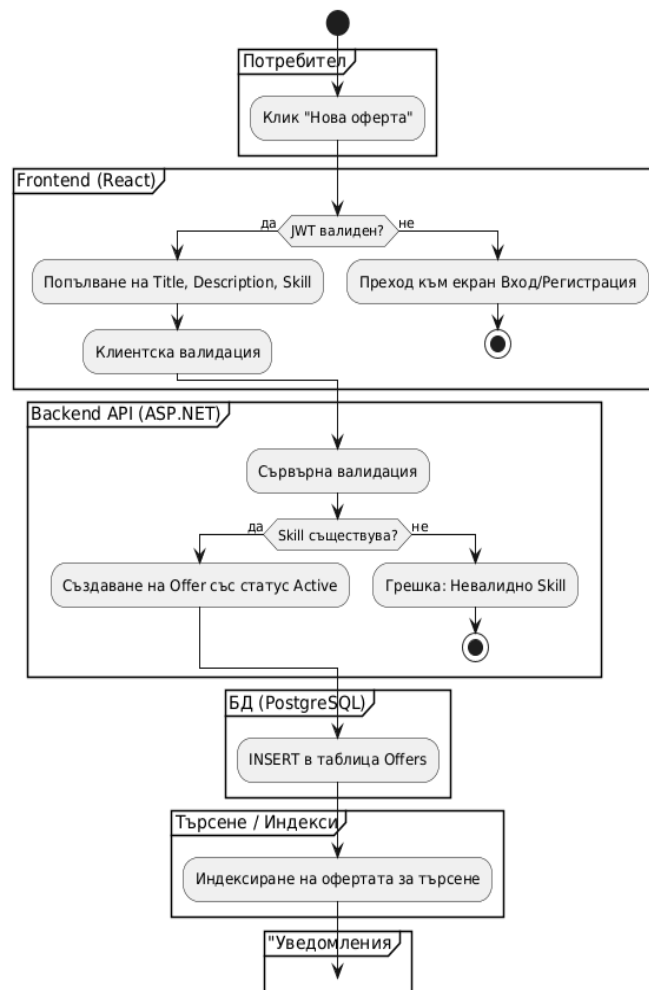
3.1. Диаграма: Регистрация и автентикация

Тази диаграма илюстрира процеса по създаване и удостоверяване на потребителски профил в системата. Представени са двата основни потока – регистрацията чрез стандартна форма и вход чрез външна идентификация (Google OAuth). Backend API изпълнява валидация на входните данни, хеширане на паролата и запис на потребителя в базата данни, след което изпраща имейл за потвърждение на акаунта. Успешната автентикация води до издаване на JWT токен за сигурен достъп, а при активирана опция се добавя и двуфакторна защита (2FA). Диаграмата демонстрира интеграцията между клиентския интерфейс, сървърната логика и външните услуги за идентификация.



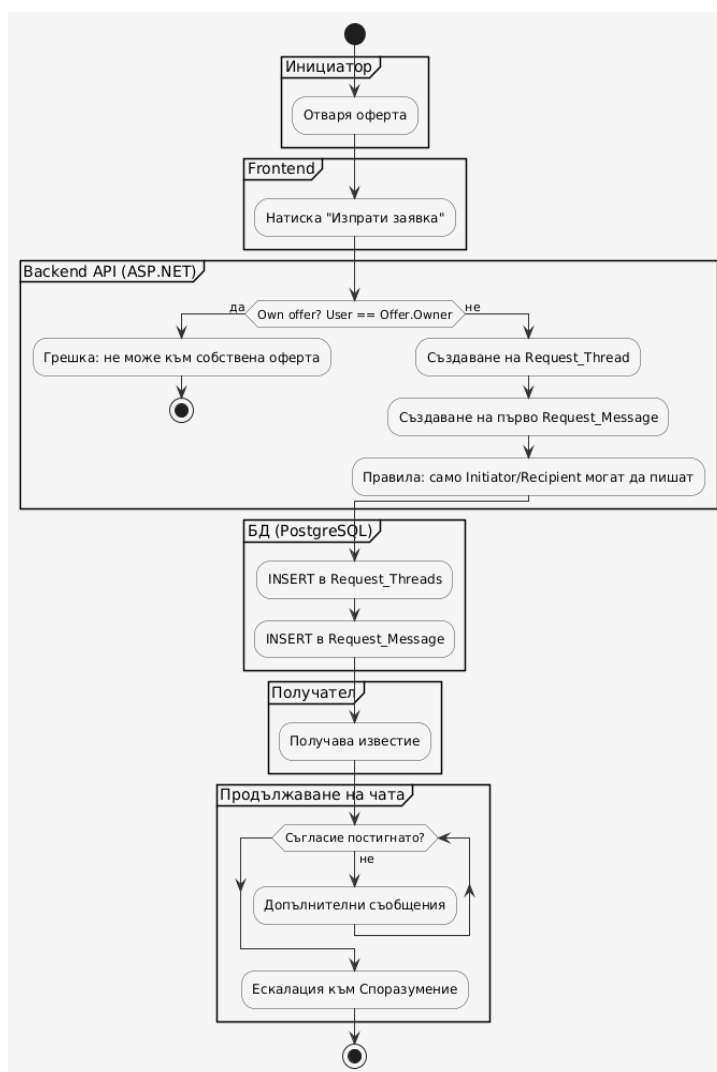
3.2. Диаграма: Създаване на оферта (Offer Creation)

Диаграмата визуализира последователността от действия при публикуване на нова оферта от регистриран потребител. Процесът започва с въвеждане на необходимата информация – заглавие, описание и свързани умения. Данните преминават през клиентска и сървърна валидация, след което Backend API създава запис в таблицата *Offers* и активира процес по индексирание за търсене. След успешно публикуване се изпраща уведомление до автора. Диаграмата акцентира върху взаимодействието между потребителския интерфейс, бизнес логиката и слоя за данни.



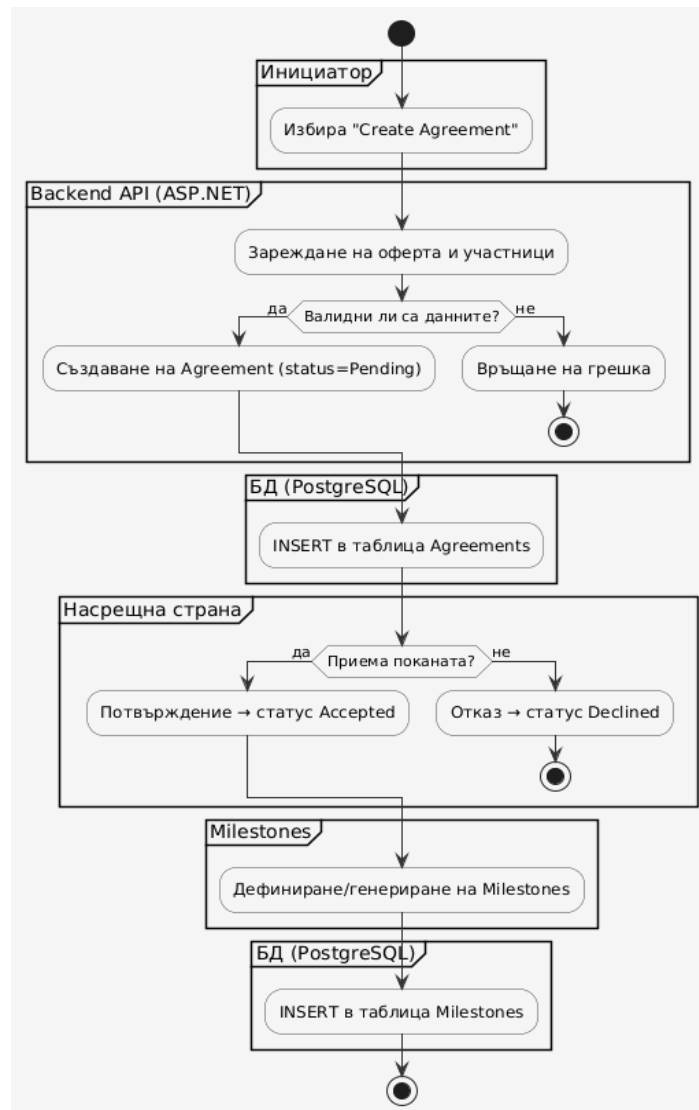
3.3. Диаграма: Създаване на заявка (Request Thread)

Тази диаграма описва комуникационния процес между двама потребители при кандидатстване по оферта. Системата проверява дали заявителят не е собственик на офертата, след което създава нова нишка за разговор (*Request_Thread*) и първоначално съобщение (*Request_Message*). Получателят бива автоматично уведомен за новата заявка. Двете страни могат да обменят съобщения, докато постигнат споразумение относно условията на бартерната сделка. Процесът служи като основа за създаване на бъдещо споразумение (*Agreement*).



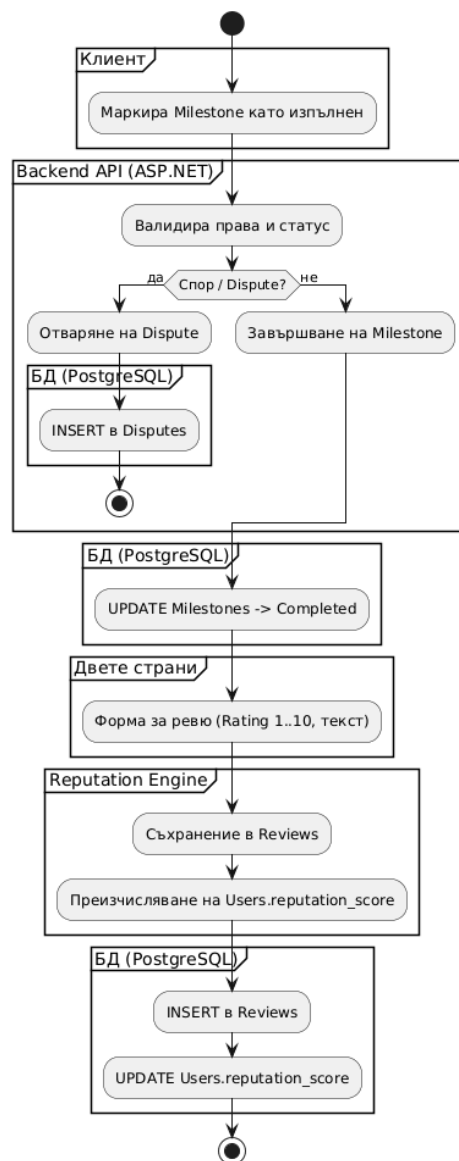
3.4. Диаграма: Създаване на споразумение (Agreement Creation)

Диаграмата представя стъпките при формиране на споразумение между участниците след успешно договаряне. Backend API валидира входните данни и създава нов запис в таблицата *Agreements* със статус *Pending*. Насрещната страна може да приеме или отхвърли поканата, като при потвърждение системата автоматично генерира свързани *Milestones*, които описват ключовите етапи и очакваните резултати от изпълнението. Диаграмата подчертава важността на валидирането, съгласуването на участниците и транзакционната последователност в базата данни.



3.5. Диаграма: Завършване на Milestone и Ревю

Последната диаграма визуализира заключителния етап от бартерния процес – приключване на Milestone и предоставяне на обратна връзка. След маркиране на Milestone като изпълнен, системата проверява за налични спорове (*Disputes*). Ако такива липсват, Milestone се отбелязва като завършен, а двете страни подават ревю с текстова оценка и рейтинг. Данните се записват в таблицата *Reviews*, след което модулът *Reputation Engine* преизчислява репутационния показател на потребителите. Диаграмата отразява взаимовръзката между процесите по контрол на качеството, доверието и устойчивата репутация в екосистемата на платформата.



4. Изглед на данните

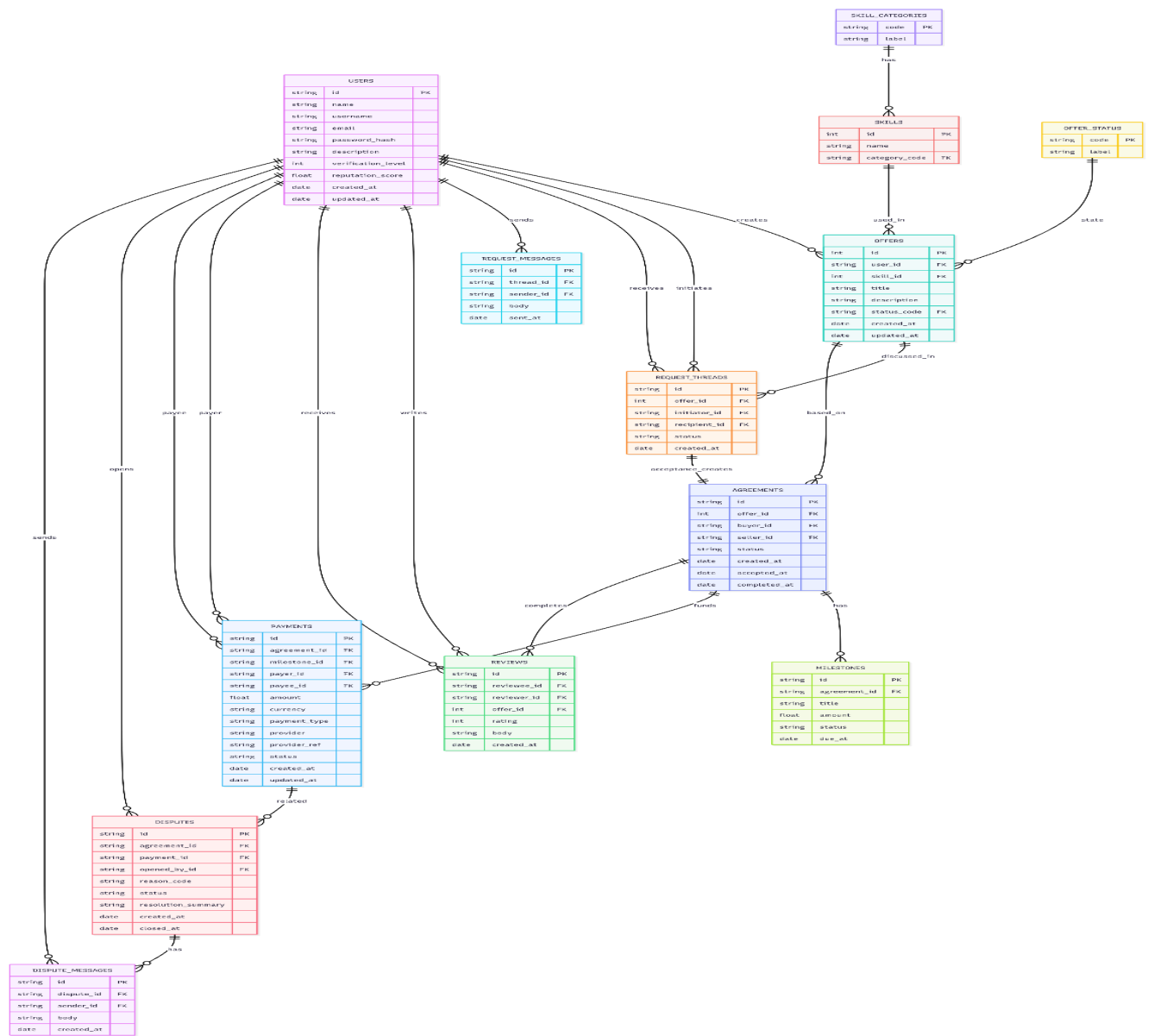


Таблица Users (потребители)

- UserId - **string/Text (Guid) - Primary Key**
- FullName - **string/Text**
- UserName - **Text (Unique)**
- Email - **Text (Unique)**
- Password_Hash - **Text**
- Description - **Text**
- Verification_level - **smallint**
- reputation_score - **numeric (5,2)**
- created_at - **timestamp**
- updated_at - **timestamp**
- Is_admin - **bool**

Таблица Skill_Categories (Категория на умения)

- Code - **text - Primary Key**
- Label - **text**

Таблица Skills

- skills_id - **text (Guid) - PK**
- Name - **text**

Таблица Offer_Status

- Code - **text (PK)**
- Label - **text**
- **Offer_Status** с Offers (One-many)

Таблица Offers

- Id - **text - PK**
- User_Id - **GUID (FK -> [users.id](#))**
- Skills_id - **GUID (FK -> [skills.id](#))**
- Title - **text**
- Description - **text**
- Status_code - **text (FK -> offer_status.code)**
- Created_at - **timestampz**
- Updated_at - **timestampz**

Таблица Request_Threads

- Id - **text/GUID**
- Offer_id - **text (Guid)** (FK -> offers.id)
- Initiator_id - **text (FK -> users.UserID)**
- Recipient_id - **Text**

Таблица Request_Message

- Id - **text/Guid**
- Thread_id - **Guid (FK -> request_threads.id)**
- Sender_id - Guid (**FK -> [users.id](#)**)
- Body - text
- sent_at - timestampz

Таблица Agreements

- Id - Guid (**PK**)
- Offer_id - Guid (FK → offers.id)
- Buyer_id - Guid (FK → users.id)
- Seller_id - Guid (FK → users.id)
- Status - text
- Created_at - timestampz
- Accepted_at - timestampz NULL
- Completed_at - timestampz NULL

Таблица Milestones

- Id - Guid (**PK**)
- Agreement_Id - Guid (**FK -> [agreements.id](#)**)
- Title - text
- Amount - numeric (12,2)
- Status - text
- Due_at - **timestampz NULL**

Таблица Reviews

- Id - **Guid** (PK)
- Recipient_id - **Guid** (FK -> users.id)
- Reviewer_id - **Guid** (FK -> users.id)
- Offer_id - **Guid** (FK -> offers.id)
- Rating - **smallint** CHECK (rating BETWEEN 1 AND 10)
- Body - **text**
- Created_at - **timestampz**

Таблица Payments

- id GUID (PK)
- agreement_id Guid (FK → agreements.id)
- payment_id Guid NULL (FK → payments.id)
- opened_by_id Guid (FK → users.id)
- reason_code text
- status text
- resolution_summary text
- created_at timestampz
- closed_at timestampz NULL

Таблица Disputes

- id Guid (PK)
- agreement_id Guid (FK → agreements.id)
- payment_id Guid NULL (FK → payments.id)
- opened_by_id Guid (FK → users.id)
- reason_code text
- status text
- resolution_summary text
- created_at timestampz
- closed_at timestampz NULL

Таблица Dispute_Messages

- Id - Guid (PK)
- Dispute_id - Guid (FK → disputes.id)
- Sender_id - Guid (FK → users.id)
- Body - text
- Created_at - timestampz
-

Таблица DisputeEvidence

Id - Guid (PK)

Dispute_id - Guid (FK → disputes.id)

FilePath - текст

Description - текст

Uploaded_at - timestampz

Таблица Notifications

Id – Guid (PK)
User_id – Guid (FK → users.id)
Title – текст
Message – текст
Type – текст
Related_entity_id – Guid NULL
Is_read – boolean
Created_at – timestampz
Read_at – timestampz NULL

Таблица Proposals

Id - Guid (PK)
Offer_id - Guid (FK → offers.id)
Proposer_id - Guid (FK → users.id)
Offer_owner_id - Guid (FK → users.id)
Terms - текст
Proposer_offer - текст
Deadline - timestampz
Status - текст
Pending_response_from_user_id - Guid NULL (FK → users.id)
Modification_count - integer
Last_modified_by_user_id - Guid NULL (FK → users.id)
Last_modified_at - timestampz NULL
Decline_reason - текст
Created_at - timestampz
Accepted_at - timestampz NULL
Agreement_id - Guid NULL (FK → agreements.id)

Таблица Proposal_History

Id - Guid (PK)

Proposal_id - Guid (FK → proposals.id)

Actor_id - Guid (FK → users.id)

Action - text

Timestamp - timestamptz

Таблица Deliverables

Id - Guid (PK)

Agreement_id - Guid (FK → agreements.id)

Title - text

Description - text

Status - text

Delivered_at - timestamptz NULL

Created_at - timestamptz

Таблица Penalties

Id - Guid (PK)

User_id - Guid (FK → users.id)

Agreement_id - Guid (FK → agreements.id)

Dispute_id - Guid NULL (FK → disputes.id)

Amount - numeric (12,2)

Currency - text

Reason - text

Status - text

Created_at - timestamptz

Charged_at - timestamptz NULL

Релации:

- Skill_Categorie <--> Skills: one-to-many
- Skills <--> Offers: one-to-many
- Offer_Status <--> Offers: one-to-many

- Users <--> Offers: one-to-many
- Offers <--> Request_Threads: one-to-many
- Users → Reviews (reviewer / recipient): one-to-many (per role)

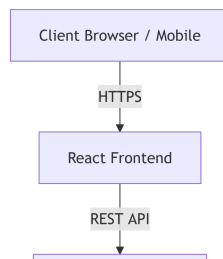
5. Изглед на внедряването – представя хардуерните ресурси (сървъри, потребителски работни станции, мобилни устройства и др.), върху които се изпълняват описаните процеси, както и начина на комуникация между тях (локална мрежа, Интернет и др.);

Общ преглед:

Платформата се реализира като **уеб-базирано приложение** с архитектура тип **client-server**, разделена на **frontend**, **backend** и **база данни**. Основните компоненти комуникират чрез **REST API** върху сигурен канал (HTTPS).

Компоненти:

- **Клиентски слой (Frontend)** – реализиран чрез React.
Потребителите имат достъп до приложението през уеб браузър (Chrome, Firefox, Edge, Safari) от настолни компютри и мобилни устройства.
Основни функции: регистрация, търсене на оферти, комуникация и управление на профил.
- **Сървърен слой (Backend)** – реализиран с **ASP.NET 9/10**, който осигурява API за достъп до бизнес логиката и базата данни.
Работи върху **Linux сървър (Ubuntu/Debian)** с минимум **4-ядрен Intel i5** и **8GB RAM**.
Използва **JWT токени** за удостоверяване и **HTTPS** за сигурна комуникация.
- **База данни – PostgreSQL**, хоствана на същия или отделен сървър, в зависимост от натоварването.
Данните за потребители, умения, оферти, споразумения и ревюта се съхраняват и индексират за оптимално търсене.
- **Интеграции** – външни API за **Google OAuth**, **SendGrid/Gmail SMTP** и бъдеща интеграция с **PayPal** или **LinkedIn**.



Мрежова комуникация:

Клиентите достъпват приложението през интернет, като заявките към API минават през HTTPS.

Сървърът може да бъде разположен в облачна инфраструктура (например AWS или Azure).

Файлове (като снимки и документи) се съхраняват в отделен storage bucket (например S3 storage).

Нефункционални изисквания

Тук се описва как избраната архитектура реализира атрибутите на качеството чрез употреба на различни тактики.

В зависимост от реализацията могат да се разгледат атрибути на качеството като:

1. Достъпност и сигурност

Рискове от откази и пробиви

- **Срив на сървър / приложение** - Възможен е необработен софтуерен бърг или претоварване, които да доведат до падане на бекенд сървъра. В резултат услугата става недостъпна за всички потребители, докато сървърът не бъде рестартиран или проблемът - отстранен.
- **Загуба на данни** - Повреда в базата данни може да доведе до загуба на важна информация (напр. потребителски профили, оферти, отзиви). Това може да се случи при хардуерен отказ, софтуерна грешка при запис.
- **Злонамерен достъп (пробив в сигурността)** - Атаки от трети лица могат да компрометират системата. Примери: SQL инжекция чрез необезопасени входни данни, XSS атаки в клиентската част, опити за неототоризиран достъп до чужди акаунти, или кражба на идентификационни токени. Подобни пробиви могат да доведат до изтичане на лични данни или поемане на контрол над потребителски профили.

- **Отказ на услугата (DoS)** - Злонамерен потребител или бот може умишлено да изпрати огромен брой заявки (или големи по обем заявки) към системата, целейки да я направи бавна или недостъпна за нормалните потребители. Макар и по-малко вероятно за малък проект, този риск съществува.

Тактики за откриване на проблеми

- **Логване на грешки и събития** – Бекендът записва подробни логове за важни събития и грешки. При всяка заявка се регистрира статуса (напр. 200 ОК или 500 грешка), а при възникване на изключение се записва лог. Анализът на лог файловете позволява на екипа бързо да засече срив в приложението или необичайно поведение.
- **Валидация на входни данни** – На всяко ниво от системата има проверки за валидност на данните (напр. формати на имейл, задължителни полета, допустими размери на текстове). Това помага да се откриват аномалии или злонамерени данни още при вход, предотвратявайки по-нататъшни проблеми (например SQL инжекции или крашове от неочаквани данни).
- **Мониторинг на ключови показатели (на базово ниво)** – В рамките на ограниченията на проекта няма да има сложна мониторинг система, но могат да се следят основни показатели: време за отговор на сървъра, натоварване на процесора/паметта и размер на базата данни. Това може да става чрез прост скрипт или периодична проверка на състоянието. Ако някой показател излезе извън норма (например паметта привършва, или отговорите забавят драстично), екипът ще бъде алармиран да провери за проблем.

Тактики за възстановяване на работоспособността

- **Рестартиране и устойчивост на сървър** - При срив на приложението, първа стъпка е незабавно рестартиране на сървърната услуга (автоматично или ръчно). Тъй като бекендът е статически уеб сървър без запазване на състояние в паметта (статусът се пази основно в базата данни и в JWT токени на клиента), рестартирането не води до загуба на данни - потребителите могат да продължат работа след кратко прекъсване. Ако проблемът е в новопуснатата версия на кода, екипът има възможност бързо да върне предишна стабилна версия от версия контрол или да приложи спешен пач.
- **Резервни копия на базата данни** - За предпазване от загуба на данни, базата данни се архивира редовно. Например, генерират се ежедневни резервни копия (backup) на всички ключови таблици. В случай на повреда

или загуба на данни, последният стабилен backup може да бъде възстановен на нов екземпляр на базата. След възстановяване, приложението се конфигурира да използва възстановената база. Екипът допълнително извършва ръчни проверки на целостта на данните след възстановката (например, проверка броя на потребителите, офертите и др. спрямо очакваното).

- **Възстановяване при пробив в сигурността** - Ако бъде открит злонамерен достъп или експлойт, реакцията включва няколко стъпки: (1) **Изолиране** - временно се спира засегнатата функционалност или се превключва услугата в режим на поддръжка, за да се предотвратят по-нататъшни щети. (2) **Анализ и корекция** - екипът идентифицира уязвимостта (напр. уязвим ендпойнт) и прилага **корекция** (поправка на кода, затягане на правата, филтриране на входните данни). (3) **Възстановяване на данни** - ако атаката е довела до манипулация или изтриване на данни, системата се връща към коректно състояние чрез backup или ръчни корекции на базата. (4) **Отмяна на компрометирани сесии** - например, анулиране на всички активни JWT токени (изискване всички да влязат отново) и принудителна смяна на пароли за засегнатите акаунти, ако има подозрение за откраднати пароли. След тези стъпки системата постепенно се връща онлайн.
- **Тестване след възстановяване** - Независимо от вида на инцидента, след възстановяване екипът извършва кратки тестове на основните функционалности (вход, публикуване на оферта, заявка, и пр.) за да се увери, че системата работи правилно и безопасно. Едва след това услугата продължава нормална работа.

Тактики за превенция на проблеми

- **Използване на HTTPS** - Цялата комуникация между клиент и сървър минава през шифрован канал (TLS/SSL). Това предотвратява атаки тип „Man-in-the-middle“ и гарантира, че чувствителни данни (напр. пароли, токени) не пътуват в открит вид по мрежата.
- **Защита на автентикацията** - Паролите на потребителите се **хешират** (със сол) в базата данни, което защитава от разкриване дори при пробив. Използват се **JWT токени** с подходящ срок на валидност и подпис, като се проверява валидността им при всяка заявка (предотвратява достъп с откраднат или подправен токен). При множество неуспешни опити за вход от един акаунт се обмисля заключване или временно блокиране.
- **Валидация и санитизация** – Всички входни данни, получени от потребителите, преминават през строги проверки и санитизация. Това означава премахване или ескейпване на опасни символи и код, с което се предотвратяват SQL Injection атаки и Cross-Site Scripting (XSS). Например,

при заявки към базата се използват параметризирани заявки или ORM, вместо динамично конкатенирани SQL низове. Полета като описания на оферти или съобщения се филтрират за скриптове, за да не се допусне XSS при визуализация.

- **Контрол на достъпа** – Back-end модулите проверяват правата на потребителя за всяко действие. Например, само автентикиран потребител може да изпраща заявка по оферта, само авторът на офертата може да я редактира или изтрива, и т.н. Това предотвратява злоупотреби като неоторизирано изменение на чужди данни.
- **Редовни backup и тестове за възстановяване** – Освен че се пазят резервни копия, екипът периодично упражнява възстановяване от backup в тестова среда, за да гарантира, че процедурата работи и backup файловете не са повредени. Това е превантивна мярка срещу изненади по време на истинска авария.
- **Прости мониторинг проверки** – Макар да не се внедрява сложна мониторинг система, на ниво инфраструктура има основни аларми (напр. ако процесорът на сървъра е натоварен 100% продължително или свободното дисково пространство падне под праг). Тези предупреждения дават шанс на администраторите да реагират превантивно – например да рестартират услуга с memory leak преди да е crash-нала, или да разширят дисковото пространство преди да се изчерпа.

Сценарии за възстановяване при проблеми

Накратко описани са няколко реалистични сценария за възникване на проблем и стъпките за реакция, които връщат системата към нормална работа:

1. **Сценарий: Срив на приложението след деплой на нова версия.** След пускане на нова функционалност, сървърът започва неочаквано да хвърля грешки и да спира. Екипът забелязва множество грешки (500) в логовете. **Реакция:** Веднага връща приложението към предишната стабилна версия (rollback) и рестартира сървъра. Услугата се възстановява в рамките на няколко минути. След това разработчиците анализират логовете, откриват причината (напр. null pointer exception в новия код) и отстраняват бъга, преди отново да деплойнат актуализацията.
2. **Сценарий: Повреда в базата данни и загуба на данни.** Поради отказ в дисковата система, базата данни спира и част от данните за последния ден са повредени. **Реакция:** Администраторът веднага преминава към аварийно възстановяване - пуска се нов екземпляр на дбто и се възстановява последният работещ backup (от предходната нощ). Услугата е недостъпна за няколко часа, през които потребителите виждат съобщение за техническа

поддръжка. След възстановяване приложението отново тръгва и екипът информира потребителите, че последните 24 часа данни може да липсват или да не са запазени коректно. Чрез комбинация от логове и потребителски обратна връзка, някои от загубените оферти/отзиви ръчно се въвеждат отново, за да се минимизира ефектът. Впоследствие се предприемат стъпки за предотвратяване на подобна повреда (напр. допълнителен диск или cloud база с висока устойчивост).

3. **Сценарий: Злонамерен пробив и XSS атака.** Забелязва се, че в описанието на една оферта злонамерен потребител е вмъкнал `<script>` таг, който изпраща бисквитките на външен сайт - класическа XSS атака. Няколко потребители докладват странно поведение. **Реакция:** Екипът незабавно деактивира показването на подозрителната оферта (сваля я офлайн временно) и анализира случая. Открива се пропуск: липсващо ескейпване на HTML при визуализация на описанието. Бързо се пуска фикс: добавя се санитизация на всички описания (превантивно за всички оферти) и опасният скрипт се неутрализира. Като допълнителна мярка, всички активни сесии са презастраховани - потребителите биват принудени да презаредят приложението, за да получат нов, валиден токен (предотвратява ползване на евентуално откраднати сесии).
4. **Сценарий: Отказ на услугата поради множество заявки (DoS).** Внезапно сайтът става изключително бавен - зловреден бот генерира хиляди заявки в минута към API-то. **Реакция:** Основният уеб сървър автоматично засича необичайния трафик (чрез проста проверка на брой заявки) и временно блокира IP адреса на атакуващия. Натоварването спада и услугата се нормализира без да е спирана изцяло. Екипът преглежда логовете, потвърждава източника на атаката и добавя правило във файъруола за трайно блокиране. Предприето е решение да се имплементира по-строга rate limiting политика за API заявките, за да не може един клиент да генерира толкова трафик в бъдеще.

2. Разширяемост – анализират се възможностите за промяна на софтуера и цената свързана с тях. Използваните тактики включват употреба на архитектурни шаблони, технологии и похвати на проектиране;

Разширяемостта на *Skills Barter* е осигурена чрез използването на **модулна архитектура** и ясни **слоеве на отговорност**. Това позволява лесно добавяне на нови функции или интеграции без нужда от сериозни промени в основния код.

Тактики и архитектурни решения:

1. Слоеста архитектура (Layered architecture)

- Разделяне на логиката на нива: *Presentation (React)*, *Business Logic (API)*, *Data Access (PostgreSQL)*.
- Всеки слой комуникира само с непосредствено съседния, което позволява частична подмяна или актуализация.

2. RESTful API с ясни интерфейси

- Нови модули могат да бъдат внедрени чрез нови крайни точки без прекъсване на текущите услуги.

3. Използване на ORM (Entity Framework)

- Осигурява лесна миграция на базата данни и добавяне на нови таблици и релации.

4. Модулност и микросервизни подходи (при бъдещо развитие)

- Приложението може да бъде разделено на микросервиси (User Service, Offer Service, Review Service), които се развиват независимо.

5. Docker и контейнеризация

- Всеки компонент може да се контейнеризира, което позволява лесно добавяне или скалиране на нови модули чрез Docker Compose/Kubernetes.

6. Гъвкавост при интеграции

- Отворени API интерфейси позволяват интеграция с външни системи – социални мрежи, платежни платформи, аналитични услуги.

3. Производителност – анализират се ресурсите, необходими за постигане на желаната производителност, при различни сценарии. Използваните тактики включват ограничаване на натоварването, планиране, репликиране на системите и др.;

Цел: стабилно време за отговор и предвидимо поведение при ръст на трафика.

- **Базови цели:** публични GET заявки $p95 \leq \sim 300\text{--}400\text{ ms}$; записи $p95 \leq \sim 500\text{ ms}$; нотификации в реално време $\leq \sim 2\text{ s}$.
- **Ограничаване на натоварването:** rate limiting на API (Nginx/ASP.NET middleware), circuit breaker за „тежки“ заявки, опашки за фонове задачи (Hangfire/Hosted Services).
- **Кеширане:** Redis за краткотрайни списъци/филтри и конфигурации; HTTP кеш с ETag/Cache-Control; CDN за изображения/статика.
- **База данни (PostgreSQL):** индекси по често ползвани полета (напр. offers(status_code, skill_id, created_at) и FTS върху заглавие/описание), винаги пагинация; избягване на N+1 заявки.
- **Скалиране:** stateless API хоризонтално зад load balancer; опция за read-replica на Postgres за тежки четения.
- **Асинхронност:** синхронният път е минимален; индексирание, имейли, нотификации – през опашки.
- **Наблюдение (минимум):** p95 латентност, error rate, CPU/RAM, DB време; аларми при отклонения.
- **Тестове:** кратки натоварващи сценарии за основните потоци (търсене, чат, създаване на оферта/сделка), „пик“ и продължителен тест.

5. Възможност за тестване – анализират се средствата и цената за провеждане тестове, с различен обхват. Използваните тактики включват налагане на тестови случаи, за удовлетворяване от софтуерните компоненти, използването на интегрирани технологии за тестване и др.;

- Автоматизация на тестване чрез frameworks за unit tests и тестове на [ASP.NET](#) разработеното API чрез Postman
- Тестване на основния път на извършването на процеса - Offer -> Request_Thread -> Request_Messages -> Agreement -> Milestones -> Payment -> Review.
- Тестова база данни със сийднати потребители, категории, умения, оферти и тестове с тях чрез докенизирана база данни.
- Ключови валидации (unit tests):
 - Потребители: уникално Username, уникален Email, не-null Password_Hash, reputation_score числово.
 - Оферти: валиден User_Id, Skills_id и Status_code (FKs трябва да съществуват).
 - Теми/съобщения: само Initiator_id или Recipient_id могат да публикуват в темата; съобщенията са подредени по sent_at.
 - Споразумения: Buyer_id != Seller_id
- Анализ на разходите (фактори и оптимизации): предпочитайте инструменти с отворен код, използваме тестови режими на доставчиците (OAuth/Tipping/SMTP), за да избегнем такси, изпълняваме задачи с голямо натоварване/сигурност по график (нощно време), за да спестяваме на разходи, минимални разходи за облак за стартиране (малък DB инстанс + съхранение на обекти).

6. Интероперабилност

- Автентикация и достъп (JWT/OAuth2): само автентикирани потребители могат да виждат своята информация свързана с уговорки и плащания
- Privacy - проверка, че личната информация на потребители като съобщения и парола се хешира и не може да бъде лесно достъпена от външни лица
- Матрица за съвместимост: браузъри (последни версии на Chrome/Firefox/Safari/Edge), мобилни прекъсвания; MIME/HTML шаблони за имейли; типове файлове за доказателства (png/jpg/pdf/mp4) с ограничения за размера.

- Локализация и часови зони: сървър в една постоянна часова зона, форматиране на клиентската локализация и съхраняване само канонични стойности

7. Използваемост

Използваемостта представлява един от ключовите атрибути на качеството в системата **Skills Barter**, тъй като платформата е ориентирана към широка потребителска база – фрилансъри, студенти и малки бизнеси. Целта е интерфейст да бъде **интуитивен, достъпен и последователен**, така че потребителите лесно да намират, публикуват и договарят услуги без нужда от предварителна техническа подготовка.

1. Принципи на използваемост

- **Простота и яснота:** Всеки екран съдържа само необходимите елементи за конкретното действие (напр. публикуване на оферта или изпращане на заявка).
- **Последователност:** Навигационното меню, бутоните и формите имат унифициран дизайн и позициониране на всички страници.
- **Обратна връзка:** След всяко действие (напр. създаване на профил, изпращане на съобщение) потребителят получава визуално или текстово потвърждение.
- **Достъпност:** Интерфейст е съобразен с потребители с различни устройства и нива на опит. Платформата се адаптира към екрани на компютри, таблети и мобилни телефони (responsive design).
- **Минимална когнитивна натовареност:** Формите и процесите са разделени на логически стъпки, което намалява риска от грешки при работа.

Компонент	Описание

Начална страница (Home Page)	Представя целта на платформата и позволява бърз достъп до регистрация и търсене на оферти.
Потребителски профил (User Profile)	Съдържа информация за умения, рейтинг, ревята и активни оферти. Дизайнът насърчава доверие и прозрачност.
Каталог на оферти (Offers Catalog)	Разделен по категории умения. Предлага филтри, тагове и търсачка за лесна навигация.
Система за съобщения (Chat / Requests)	Позволява директна комуникация между потребители. Интерфейсът поддържа история на разговорите и известия в реално време.
Управление на оферти (Offer Dashboard)	Дава възможност на потребителя да редактира, архивира или активира свои оферти.
Нотификации	Интерактивен панел за известия – нови съобщения, ревята и промени в статуси.
Административен панел (Moderator / Admin View)	Интерфейс за преглед и контрол на съдържание, отчети и сигнали от потребители.

3. Навигация и потребителско изживяване

- Основна навигация: фиксирано горно меню с раздели *Home*, *Offers*, *Messages*, *Profile* и *Help*.
- Вториична навигация: странични панели в профила и каталога с филтри (по категория, рейтинг, локация и др.).
- Интерактивни елементи: бутони с визуален фийдбек, икони с подсказки (tooltips), съобщения за грешки и успех.

- Форма за контакт и поддръжка: достъпна от всяка страница, с възможност за директна връзка с екипа.

4. Визуален и стилев подход

- Интерфейсът следва модерен flat дизайн, с пастелни цветове и висок контраст за четимост.
- Шрифтове: *Roboto* / *Open Sans* за добра четливост на всички устройства.
- Използва се адаптивна мрежова структура (grid layout), базирана на Bootstrap / Tailwind CSS.
- Икони: bootstrap icons

8. Система за комуникация и разрешаване на спорове

Общ преглед

Платформата използва известия вместо чат. Потребителите взаимодействат, като попълват формуляри и натискат бутони. Системата уведомява другата страна, когато бъде предприето действие. По този начин всички взаимодействия са структурирани и документирани.

Процес на сключване на споразумение

1. Потребител А публикува оферта (какво предлага, какво иска).
2. Потребител Б изпраща предложение с конкретни условия и краен срок.
3. Потребител А приема, променя или отхвърля. Ако бъде променено, потребител Б го преглежда отново.
4. Когато и двамата приемат, споразумението влиза в сила.
5. Всеки потребител изпраща своето изпълнение с линк и описание.
6. Другият потребител одобрява, иска преразглеждане или отваря спор.
7. Когато и двамата одобряват, и двамата оставят отзиви. Споразумението е сключено.

Доказателства

Няма качване на файлове. Потребителите предоставят доказателства, като поставят линкове (Google Drive, Dropbox, уебсайт на живо) и пишат описания. Другата страна потвърждава получаването, като кликне върху „Одобри“.

Спорове

Ако потребител не достави или достави лоша работа, другата страна започва спор. Тя избира причина и обяснява проблема. Обвинената страна има 72 часа, за да отговори. Ако не отговори, автоматично губи.

Системата изчислява резултат въз основа на това дали работата е предадена, дали е предадена навреме и дали е одобрена преди спора. Резултат от 70-100 е в полза на доставчика. Резултат под 40 е в полза на жалбоподателя. Резултатите между тях се препращат на модератор.

И двете страни могат да приемат предложеното решение или да ескалират спора до

Доказателства

Няма качване на файлове. Потребителите предоставят доказателства, като поставят линкове (Google Drive, Dropbox, уебсайт на живо) и пишат описания. Другата страна потвърждава получаването, като кликне върху „Одобри“.

Спорове

Ако потребител не достави или достави лоша работа, другата страна започва спор. Тя избира причина и обяснява проблема. Обвинената страна има 72 часа, за да отговори. Ако не отговори, автоматично губи.

Системата изчислява резултат въз основа на това дали работата е предадена, дали е предадена навреме и дали е одобрена преди спора. Резултат от 70-100 е в полза на доставчика. Резултат под 40 е в полза на жалбоподателя. Резултатите между тях се препращат на модератор.

И двете страни могат да приемат предложеното решение или да ескалират спора до

Доказателства

Няма качване на файлове. Потребителите предоставят доказателства, като поставят линкове (Google Drive, Dropbox) и пишат описания. Другата страна потвърждава получаването, като кликне върху „Одобри“.

Спорове

Ако даден потребител не достави или достави лоша работа, другата страна отваря спор. Тя избира причина и обяснява проблема. Обвинената страна има 72 часа, за да отговори. Ако не отговори, автоматично губи.

Системата изчислява резултат въз основа на това дали работата е била предадена, дали е била предадена навреме и дали е била одобрена преди спора. Резултат от 70-100 е в полза на доставчика. Резултат под 40 е в полза на жалбоподателя. Резултатите между тях се препращат на модератор.

И двете страни могат да приемат предложеното решение или да го препратят на модератор за човешка проверка.

Механизъм за налагане на санкции

Преди да сключат споразумения, потребителите трябва да добавят метод на плащане. Системата одобрява размер на санкцията, но не я начислява. Ако потребител загуби спор или се откаже от споразумение, санкцията се начислява. Това гарантира, че потребителите приемат сериозно своите ангажменти.

Наказанията се налагат, когато:

Ситуация

Резултат

Загубен спор (резултат под 40)

Налага се пълната санкция

Няма отговор на спора в рамките на 72 часа

Налага се пълната санкция

Отказано споразумение (14 дни без активност)

Налага се пълната санкция

Загубен спор (резултат 40-60)

Налага се половината санкция

Нотификации

Потърпващите получават нотификации за всяко действие: нови предложения, променени условия, предадени резултати, искания за преразглеждане, приближаващи крайни срокове и спорове. Нотификациите се появяват в приложението и се изпращат по имейл според предпочитанията на потребителя. Нотификациите за спорове винаги се изпращат незабавно.

Резюме

Потребителите публикуват оферти, изпращат предложения, подават резултати чрез линкове и одобряват или оспорват. Без чат, без качване на файлове. Системата проследява всичко автоматично. Наказанията гарантират отчетност. Този подход е лесен за изграждане, лесен за модериране и създава ясен архив за разрешаване на спорове.